

		#4463](https://github.com/kotest/kotest/issues/4463), [PR #5807](https://github.com/kotest/kotest/pull/5807)
4	JSON Matchers 测试 Json 测试 测试	[issue #4601](https://github.com/kotest/kotest/issues/4601), [PR #5795](https://github.com/kotest/kotest/pull/5795)
5	@OnlyInputTypes 测试 测试 测试 测试	[issue #5589](https://github.com/kotest/kotest/issues/5589), [PR #5789](https://github.com/kotest/kotest/pull/5789)
6	shouldHaveSingleElement 测试 测试	[issue #5755](https://github.com/kotest/kotest/issues/5755), [PR #5756](https://github.com/kotest/kotest/pull/5756)

[Armeria](https://github.com/line/armeria) — LINE 的 Java 的 RPC 框架 (5.1k stars)

问题	问题
RequestContextExporter BuiltInProperty 测试 + Logback MDC export 测试 测试 (+339/-5, 测试 1/2 测试, 测试 测试 测试 测试)	[issue #4403](https://github.com/line/armeria/issues/4403), [PR #6683](https://github.com/line/armeria/pull/6683)

[Spring Batch](https://github.com/spring-projects/spring-batch) — Spring 的 批处理 框架 (2.9k stars)

测试 测试 测试, PR 测试 测试.

问题	问题
fault-tolerant step chunk scanning 测试 测试 4测试 测试 测试 (测试 "documentation issue" 测试 测试)	[issue #3946](https://github.com/spring-projects/spring-batch/issues/3946)
conditional flow decider order 测试 (测试 "this is a bug" 测试)	[issue #4478](https://github.com/spring-projects/spring-batch/issues/4478)

[Spring Cloud Gateway](https://github.com/spring-cloud/spring-cloud-gateway) — Spring reactive API gateway

이 글은 5.1.x 버전에서 작성되었습니다.

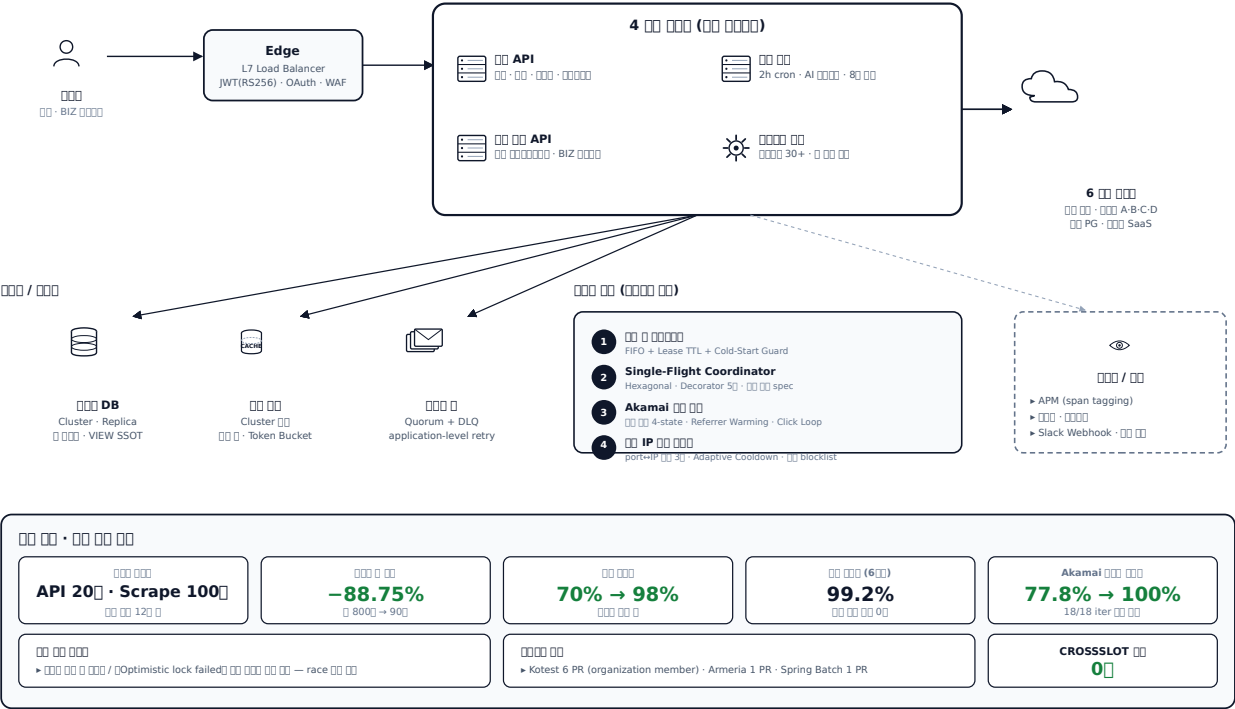
이 글	이 글
Apache HttpClient5 이 글은 이 글 (이 글은 이 글, 이 글)	[issue #3311](https://github.com/spring-cloud/spring-cloud-gateway/issues/3311)
API Versioning Predicates 이 글은 이 글 (이 글은 이 글은 이 글)	[issue #4024](https://github.com/spring-cloud/spring-cloud-gateway/issues/4024)

이 글 — 이 글(Lemong) | | | 2024.11 ~ 이 글

이 글은 이 글

A1. 이 글은 이 글 SaaS — 이 글은 이 글

4 이 글은 이 글 · 6 이 글은 이 글 · 이 글은 이 글 · 이 글은 이 글



A1. 이 글은 이 글

이 글은 이 글은 이 글 이 글 이 글 이 글 SaaS 이 글은 이 글은 이 글. 이 글 API 이 글 20 이 글, 이 글은 이 글 100 이 글, 이 글 이 글 12 이 글 이 글은 이 글, 이 글 30+ worker 가 이 글은 이 글. 이 글 4 이 글은 이 글 이 글.

이 글	이 글	이 글
이 글 API 이 글	이 글 · 이 글 · 이 글 (BIZ 이 글 ↔ 이 글)	REST API 137 controllers / 이 글 이 글

		152
200 OK	200 cron 200 200 · AI 20000 · Active-Active 8000	5,700+ 200 / 13,000+ 200
200 API	2000 200 200 · 20000 200000000 · AI 20000 200 · BIZ 20000	200 2000000 Hash + ZSet
20000 200 20000	6 200 2000 20000 · 200 200 2000000 · Akamai 200 · 200 IP 200	20000 worker 30+

200 20000 20000 200000000 · 20000 200 (Quorum Queue) · 200 200 (Cluster 200) · 20000 DB (Cluster) 가 200000000 APM (span tagging) · 20000 200000 (20000 200 20000) · Slack Webhook (2000 20000 · cron 200/200/200) 200 20000 20000 200.

2000 200·200(A) → 200 200 2000·200 2000(B) → 200 2000 200·200 200 200(C) → AI 2000000·200 2000(D) 200 2000 150 200 200 → 200 → 200 2000 2000000.

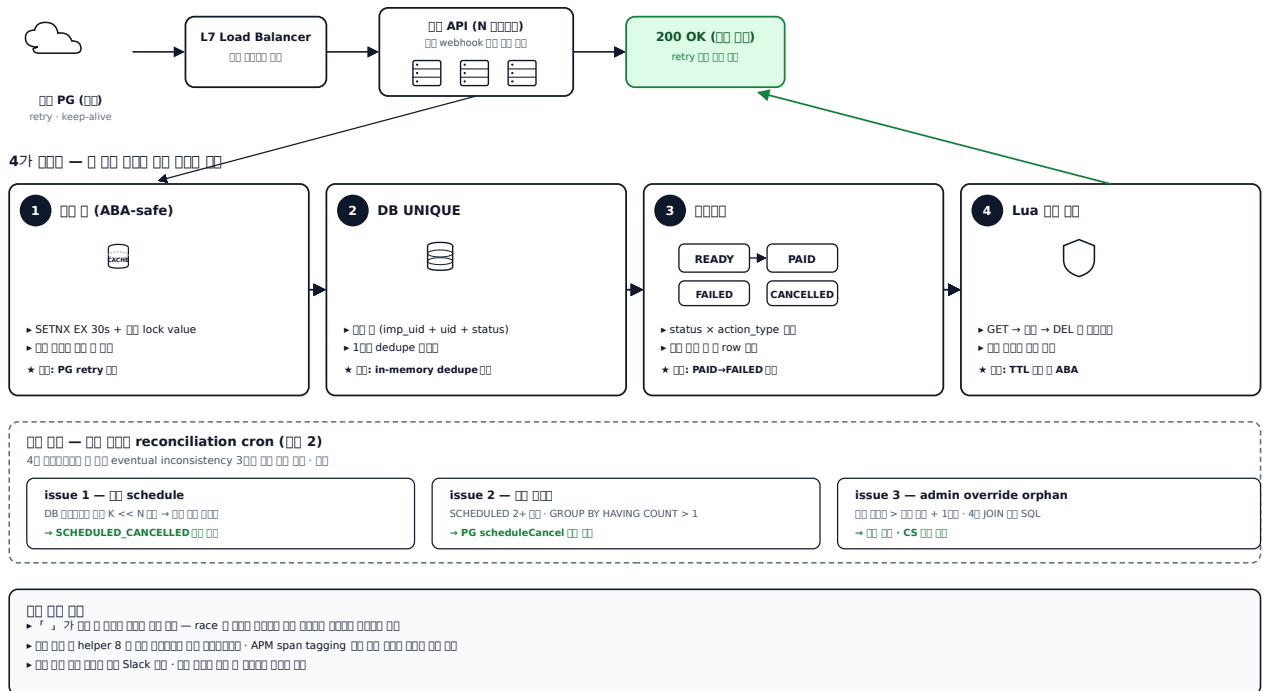
20000 A. 200·200

200 PG webhook 2000, 20000 200 200, 5 — 200 200 200 2000 20000 40 200 200 200.

20000 webhook 40 20000 — 200 200 (ABA-safe) · DB UNIQUE · 2000000 · 200 200

A2. 20000 webhook 40 20000 — 200 200 · DB 20000 · 2000000 · 200 200

race · ABA · 200 200 · out-of-order 200 4 hazard 200 20000 200 200



A2. webhook 40 200 20000

200 — 200 PG webhook 200 200 20000 200 20000 200 50.

- PG retry (5xx / timeout)
- schedule 1회 fallback retry → PAID / FAILED 후 retry
- retry 횟수 제한
- retry 간격 제한
- Load Balancer keep-alive 유지

INSERT 후 row 존재 → retry → 2회 retry → retry 횟수 초과 → DB UNIQUE 오류 발생.

in-memory dedupe 처리. ORM findOne → save read-modify-write race window 발생. retry 횟수 제한 TTL 및 ABA 처리(Martin Kleppmann Redlock 참고) DB UNIQUE 후 PG retry 1회 retry. INSERT race 처리.

— race 발생 시 4회 retry 후 retry 횟수 초과.

1. **ABA-safe** — SETNX EX 30s + lock value · Stripe Idempotency Key 후 lock value ownership
2. **DB UNIQUE** — imp_uid + merchant_uid + status · 1회 dedupe window · fail-open 후 retry
3. 상태 — READY → PAID / FAILED / CANCELLED × action_type 후 (PAID 후 FAILED 후) row 존재
4. **Lua** 후 GET → DEL 후 retry · TTL 및 ABA 처리

SETNX + Lua DEL helper 후 8회 retry(promotional upgrade · change plan · verify code · verify code) helper 후 retry.

후

- 7회 retry 후 race 발생 후 retry 횟수 초과.
- Slack 후 — retry 횟수 초과.
- APM span tagging 후 (amount_mismatch, coupon_select) 후 retry.
- helper(delIfValueMatches)가 8회 retry(promotional upgrade · change plan · verify code · verify code) helper 후 retry.

JVM/Spring ([commerce-comment-platform-be](https://github.com/PreAgile/commerce-comment-platform-be))

4회 retry Java 21 / Spring Boot / JPA / Redisson 후 **ADR-006** 후 4회 retry.

- ① Redisson watchdog ([ADR-004](https://github.com/PreAgile/commerce-comment-platform-be/blob/main/docs/adr/ADR-004-distributed-lock.md)) — Redisson vs SET NX+Lua 후 retry
- ② DB UNIQUE — JPA @Version + OptimisticLockException + MySQL ER_DUP_ENTRY → @RestControllerAdvice 후
- ③ idempotency_key 후 + 4회(RECEIVED→PROCESSING→SUCCESS/FAILED)
- ④ @Transactional(REQUIRES_NEW) + OSIV=false ([ADR-BE-008](https://github.com/PreAgile/commerce-comment-platform-be/blob/main/docs/adr/ADR-BE-008-transaction-boundary.md)), [EXP-09b 9회 retry 후 retry]

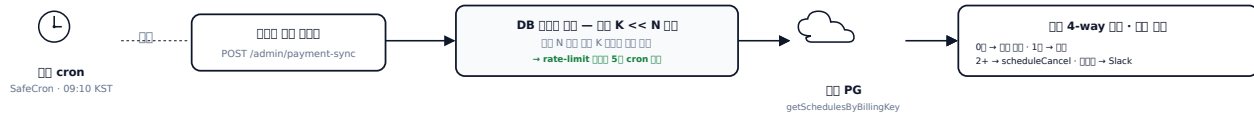
](https://github.com/PreAgile/commerce-comment-platform-be/blob/main/docs/experiments/EXP-09b-tx-boundary.md))

EXP-02 0000 00 0 40 (0000 · 0000 · GET_LOCK · Redisson) 00 00 — GET_LOCK 00 00.

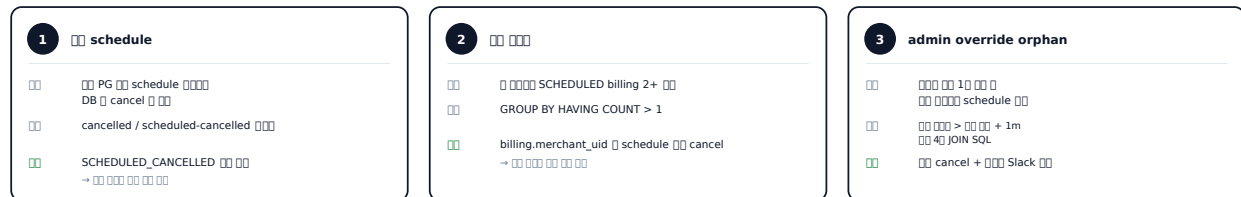
00 0000 **reconciliation** — 00 **schedule** · 00 0000 · **admin override orphan** 00 00

A3. 00 0000 reconciliation — 30 inconsistency 00 00 · 00

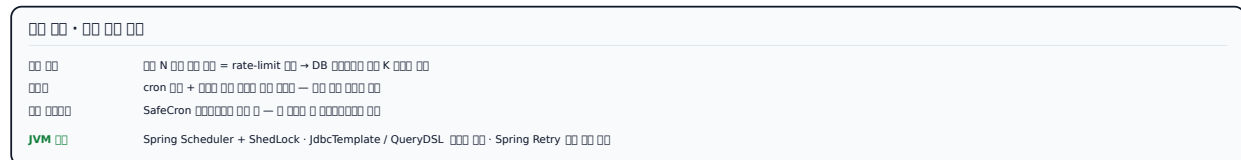
00 00-CS 0000 00 0000 00 · DB 0000 K << N 00 · 4-way 00 00 00



30 inconsistency — 0000 0000 0 00 00



00 — 00 0000 00 0000 00



A3. 00 0000 reconciliation — 30 inconsistency 00 00

00 — 00 1 0 40 0000000 0 00 300 silent 0000 0000 000000.

- 00 **schedule** — 00 PG 00 schedule 0 000000 DB 0 cancel 0 00 → 00 0000 00 00
- 00 0000 — 0 0000 SCHEDULED billing 2 0 00 00 00 0000 00 00
- **admin override orphan** — 0000 0000 10 0000 00 000000 00 000000 schedule 00 → active 00 0 0 00 0 0 0000 0000 / 00 0000 0000 0000 000000. 00 PG API 0 00 0000 00 0000 00 0000 00 0000 000000, DB SCHEDULED 0 PG schedule_status 가 0000eventual consistency 000000.

00 — 0000 00 K << N 00 00 → 00 00 0000 → 4-way 00 00 00」 0000 00 cron + 0000 00 0000 00 000000 0 0. DB 000000 00 customer 0 0000, 0000 K 00 000000 00 PG schedule API 00, 0000 4-way 00 00 — schedule 0000 DB 0000 00, 100 00, 20 000000 00 schedule 00 cancel, API 000000 Slack 00. admin override orphan 「 0 0000 0000 00 000000 0000 100 00 00 0000 00 SQL 0 000000. 00 cron 0 SafeCron 0 0000(00 14)0 00 0000000 0 00 000000 00 0 00.

00

□□ — 「 □□ □□ / □□ □□ □□ □ **hierarchical** □□□ □□. □□ □ □□□ □□ process-local □□□□ □□□□□ □□ — (platform_id, platform_password) □□ □□□ □□ □□□ ThreadPool □ □□□□ □ □□□ □ worker thread 가 □□□□ □□□ □□ □□□□□. I/O-bound(HTTP/DB)□ thread 가 □□. □□ □□ □□ □□□□□ □□□□ □□□□ □ □□ □□□ □□ □□. □ □□ □□ □□ □ □□ □□ □□ □□ — □ □□ □□□ □□□ □□□ 100% □□ □□□□ □ □□ □□ □□ □□ □□(circuit breaker) □□.

□□

- □□ □□□ — □□ □□□ 13,127 □□□ 2h cron □□ □□.
- □□ platform_id □□ □□ **0**□ (□□ □□ □□).
- □□ □□ □□ □□□ □□□ □□, □□ □□ □□ □□ — □□ □□ □□.
- 5,000+ □□ □□ □□□□□ ordered logBuffer flush □□ □□□ □□ □□□ □ □□ □□ □ □.

JVM/Spring □□□ ([commerce-batch-orchestrator](https://github.com/PreAgile/commerce-batch-orchestrator) + [commerce-external-gateway-kt](https://github.com/PreAgile/commerce-external-gateway-kt))

- Spring Batch + **ShedLock** □□ □□ □□□□ cron □□ □□
- [ADR-MQ-007 Reader 4□ □□](https://github.com/PreAgile/commerce-batch-orchestrator/blob/main/docs/adr/ADR-MQ-007-reader-4.md) → QuerydslZeroOffsetItemReader □□ (E-MQ-04~07 100□ row □□ □□)
- [ADR-MQ-009 CooperativeStickyAssignor (KIP-429)](https://github.com/PreAgile/commerce-batch-orchestrator/blob/main/docs/adr/ADR-MQ-009-rebalance.md) — Rebalance stop-the-world □□
- Resilience4j Bulkhead.semaphore □ □□ □□ □□□ □□ + CircuitBreaker □ □□□ □□

스텝 1 - 스텔스 포크 + 가짜

스텝 1 - 스텔스 포크 + 가짜

stealth fork 30+ 개 = CPU/메모리 1 개 → 스텔스 포크 30개 + 가짜 8개 + 가짜 8개

가짜

가짜

- ▶ 1 개 = stealth fork 1 개 ≈ **1 GB+ RAM**
- ▶ launch 5~10 개, CPU 1~2 개
- ▶ worker 30+ × 6 개 = 200+ 개
- ▶ fingerprint 30개 = 30개
- 200+ 개 + OOM 개 + ASN 개
- 가짜

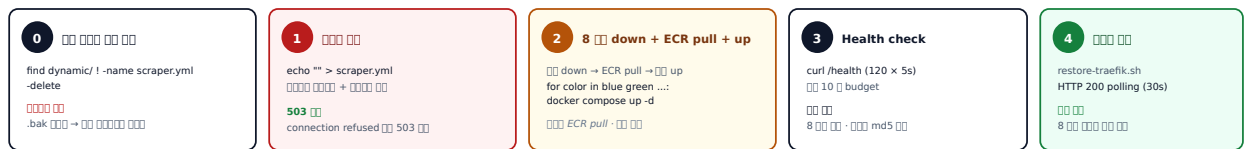
가짜

- ▶ CPU 32 개
- ▶ RAM 512 GB
- ▶ 8 개
- ▶ 4 개 · 64 GB RAM
- ▶ blue/green/.../black
- ▶ fingerprint
- ▶ 7 개
- ▶ 8 개

스텝 2 - 가짜 → 가짜 → 가짜



스텝 3 - 5-step



스텝 4 - 가짜

가짜

- ▶ 6 개
- ▶ 6 개 retry → 21 개
- ▶ 1~24 개
- ▶ 가짜

가짜

- ▶ deactivate_shops(shops_to_process, ...)
- is_active=4 개
- ▶ PLATFORM_LOGIN_ERROR_COUNTS
- ▶ A: 3 B: 2 C: 3
- ▶ 0 개

스텝 5 - 스텔스 포크 + 가짜

스텝 5 - 스텔스 포크 30+ 개 6 개

- 1 개 = 1 개 ≈ **1 GB+ RAM**, launch 5~10 개, CPU 1~2 개
- 30+ 개 × 6 개 = 200+ 개
- 200+ 개 + OOM 개 + ASN 개
- fingerprint 가

가

- 가
- retry 21 개 + 가

스텝 6 - ECS 가 + 가

가

- CPU 32 개 / RAM 512 GB
- 8 개 (4 개 · 64 GB · blue / green / yellow / purple / orange / cyan / white / black 개)
- 가가 fingerprint · network namespace · 가

스텝 7

- 2개 cron 가 fan-out → Reverse Proxy HTTP call
- traefik/dynamic/scrapper.yml 개 8개

- $\text{pages} \times \text{bytes per page} = \text{total memory size}$, $\text{total memory size} \times \text{bytes per word} / \text{word size}$
- $\text{page} \times \text{OOM} / \text{page} = \text{total 700 MB}$ (total memory size)

□□□ □□□ 5-step:

1. 백엔드 서버 배포 — .bak 백엔드 서버 (Reverse Proxy 가 백엔드 서버 배포 → 백엔드 서버 배포 후 백엔드 서버 배포)
2. 백엔드 서버 — scraper.yml 백엔드 (백엔드 서버 배포 + 백엔드 서버, connection refused 가 백엔드 503 백엔드)
3. 8 백엔드 서버 down + ECR pull + 백엔드 up
4. Health check — /health 120 × 5s 백엔드 (백엔드 10s budget · 백엔드 md5 백엔드 백엔드)
5. 백엔드 서버 — restore-traefik.sh + HTTP 200 polling 30s 백엔드 8 백엔드 백엔드 백엔드

□□ □□ □□ □□□ □□:

- `_deactivate_shops` `is_active=4`
- `dict` — `0` (=), 2~3 ()
- — `00.가`

11

- 200+ `npm install` 실패 후 `rm -rf node_modules`
- `npm run build` OOM 발생하면 7번 정도 → `npm cache clean --force`
- `npmqscript.sh` 이 8 번 정도 실패하면 — `npm cache md5` 확인
- `npm run build` — `npm run test`도 실패하면, `npm cache clean --force`
- 깃 — `npm run build` worker 14 → 30 → 25 정도

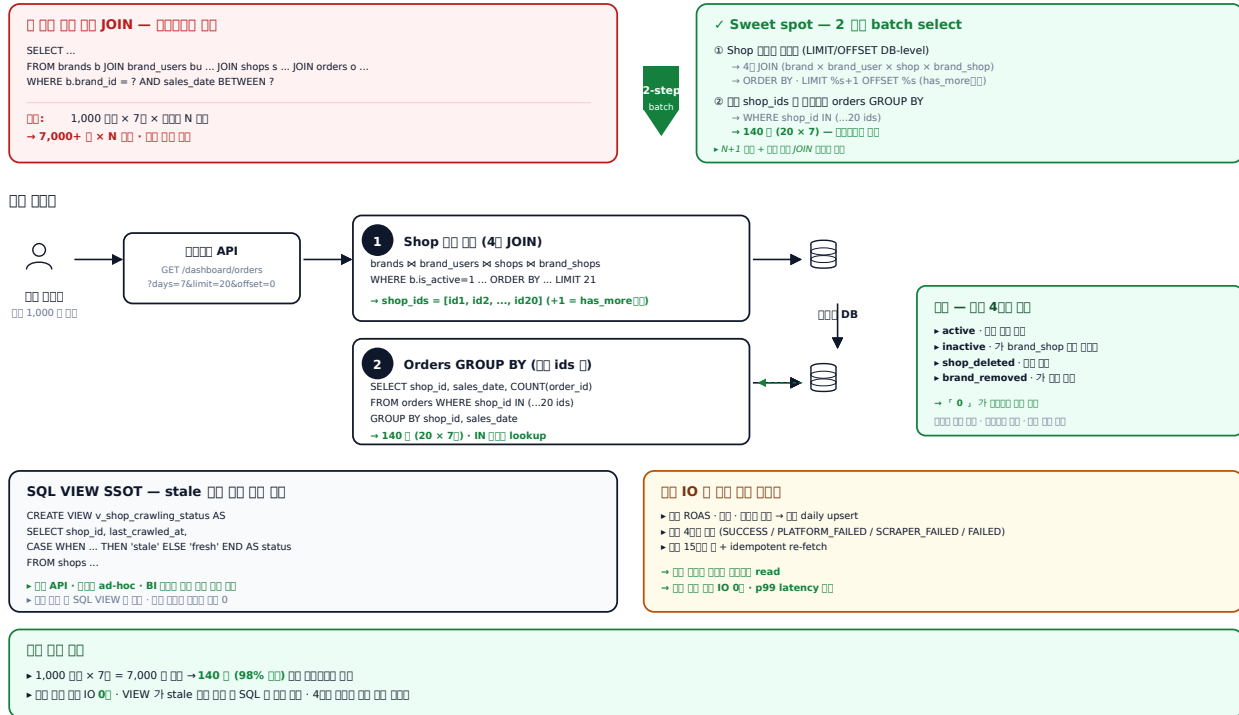
JVM/Spring  ([commerce-external-gateway-kt](https://github.com/PreAgile/commerce-external-gateway-kt))

- Spring Boot Actuator /health + readinessProbe / livenessProbe + Micrometer
- Resilience4j Bulkhead (Semaphore vs FixedThreadPool) —
- Resilience4j CircuitBreaker dict — ADR-007 50%/70%/90% 70%
- K8s rolling update + maxSurge/maxUnavailable (rolling update)

BIZ 1,000 × 7 - 2 + DB-level pagination + SQL VIEW SSOT

B3. BIZ - 2 + SQL VIEW SSOT + IO

1 → 1,000 × 7 - N + 1 JOIN sweet spot



B3. BIZ 2 + SQL VIEW SSOT

BIZ 1,000 × 7 - 2 + DB-level pagination + SQL VIEW SSOT. 1,000 × 7 = 7,000 (98%) → 140 (20 × 7). stale SQL VIEW · API · ad-hoc · BI · daily upsert + 400 · read · IO · p99 latency.

「select shape N+1 JOIN sweet spot 2-step batch select + SQL VIEW SSOT + IO」. 1. shop (brand × brand_user × shop × brand_shop 4 JOIN, LIMIT +1 has_more DB-level pagination), 2. shop_ids ~20 orders GROUP BY (140 = 20 × 7). 4 (active / inactive / shop_deleted / brand_removed) · 0 · stale SQL VIEW · API · ad-hoc · BI · daily upsert + 400 · read · IO · p99 latency.

1

- 1,000 × 7 - 2 + DB-level LIMIT/OFFSET - IO 0.
- VIEW stale SQL.
- 1500 + idempotent re-fetch.
- 「raw SQL N+1 JOIN sweet spot 2-step batch select + SQL VIEW SSOT + IO」.

JVM/Spring (commerce-comment-platform-be)(https://github.com/PreAgile/commerce-comment-platform-be))

- Spring Data JPA + **QueryDSL** / dynamic order by ([ADR-BE-009 No-Offset Cursor Pagination](https://github.com/PreAgile/commerce-comment-platform-be/blob/main/docs/adr/ADR-BE-009-no-offset.md))
- [EXP-07 1M row OFFSET 171ms → Cursor (https://github.com/PreAgile/commerce-comment-platform-be/blob/main/docs/experiments/EXP-07-pagination.md) — row constructor
- @Subselect SQL VIEW + @EntityGraph N+1 ([EXP-04 N+1 4-depth 121 prepStmt → JOIN FETCH 1](https://github.com/PreAgile/commerce-comment-platform-be/blob/main/docs/experiments/EXP-04-n-plus-one.md))

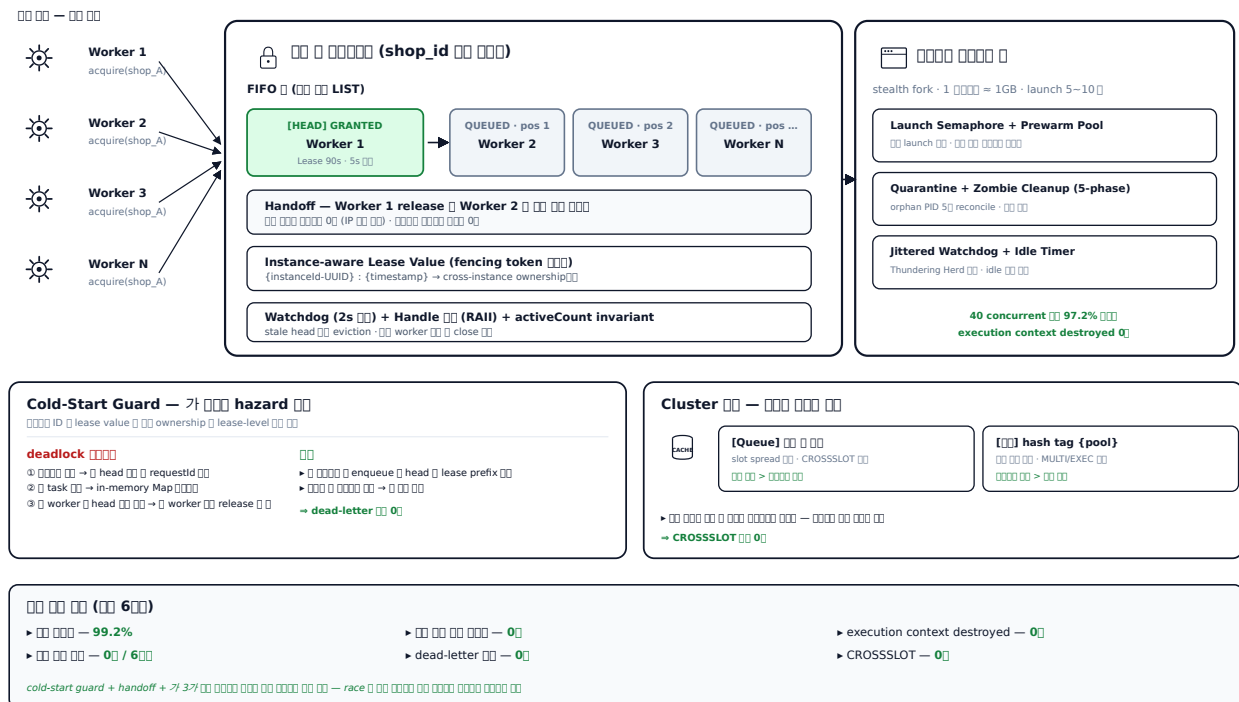
C. .

6 · race · IP · Akamai · single-flight coalescing · fencing token · Port-Adapter

FIFO + lease TTL + cold-start guard 30+ worker race

C1. FIFO + Lease TTL + Cold-Start Guard

30+ race 4 layer Handoff + 3



C1. FIFO + Cold-Start Guard

100 — 100 30+ 100 100 consumer 100 10000. 100 (shop_id) 100 API(getReviews, addReply, getStores, keepAlive)가 100 100 10000 10000 10000 100 worker 가 10000 10000 10000. 100 100 100 page 100 100 navigation 100 execution context destroyed, 100 100 100 100 100 100 IP 100 100 100(1~24가나 100 100 100 100). 10000 10000 100 mutex 100 10000 10000 — 10000 10000 1 10000 $\approx 1\text{GB}+$, launch 5~10 → race 100 N 100 10000 100. 10000 10000 in-process Map 10000 10000, 100 100 Cluster 10000 100 10000 100 100 CROSSLOT 10000 10000. 가 10000 hazard 100 **cold-start deadlock** — 가 10000 10000 in-memory Map 100 100 10000 100 100 head 100 100 10000 requestId 100, 10000 worker 가 100 head 10000 10000 100 worker 100 10000 release 100 10000.

100 — **4 layer** 10000: ① FIFO 100(100 100 LIST) ② per-request lease(90s TTL, 5s 100) ③ **instance-aware lease value** = {instanceId-UUID} : {timestamp} (Martin Kleppmann 100 fencing token 10000 10000 — 10000 10000 lease value 100 100 cross-instance ownership 100 lease-level 100 100) ④ handoff pattern (release 100 100 entry 가 가 = 100 10000). Cluster CROSSLOT 100 10000 100 100 100 — queue 100 100 100 100 100(100 100 100), reputation(100 100) hash tag {pool} 100 100 100(10000 100 100). 100 10000 100 100 10000 10000 가 100 10000 100 100 10000 100. attach/release Handle 100(RAII)100 activeCount invariant 100 10000, pendingClose 5 100(immediate / defer / idle-timer / hasWaitingRequests handoff / release)100 100 worker 100 10000 100 close 100 100, 100 release 10000 close. 1000000 forceRelease(100 kill switch), forceTerminate(activeCount 100 0), evictStaleHead(2 100 head 100 lease 100 100 LREM) 3.

100

- 100 10000 **99.2%** (100 600).
- 100 100 100 **0** / **6** — 100 100 + Idempotency-Key 100.
- execution context destroyed 100 **0**.
- 10000 10000 100 dead-letter 100 100 **0** — cold-start guard 100 100.
- CROSSLOT 100 **0** — hash tag / 100 100 10000 100 100 100.
- 100 100 100 100 **0**.

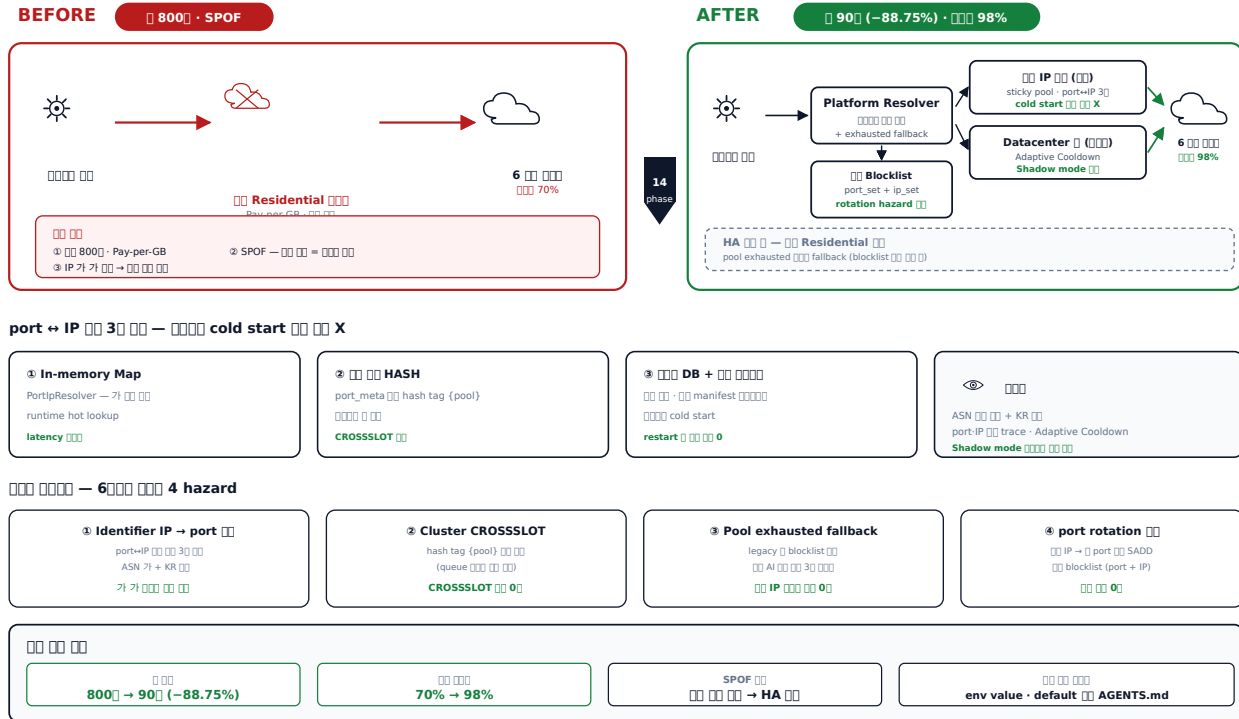
JVM/Spring 10000 ([commerce-external-gateway-kt](https://github.com/PreAgile/commerce-external-gateway-kt))

- [ADR-SCR-009 3-Layer 100 100
100](https://github.com/PreAgile/commerce-external-gateway-kt/blob/main/docs/adr/ADR-SCR-009-session-cache.md) — Memory L1 → Redis L2 → 100 100 L3
- Kotlin Coroutines Mutex + withLock per-key 100 in-process FIFO layer
- Redisson RFairLock (FIFO + lease auto-extend) + @PreDestroy graceful release
- Micrometer gauge 100 100 / 100 hit rate 100 (SES-1 300 hit rate 100)

IP pool size — pool size + size 88.75% (800 → 90)

C2. IP pool size — pool size · size 88.75% · size 70% → 98%

14 phase phased rollout — port→IP 3 · Adaptive Cooldown · blocklist · pool exhausted fallback



C2. IP pool Before/After

— Residential pool size / SPOF / pool size hazard 가

- / **SPOF** — 800 · =
 - **Identifier** — Datacenter port IP 가 가 → stable identifier(port)가
 - **Cluster CROSSLOT** — · STREAM · blocklist MULTI/EXEC
 - **port rotation** — IP 가 port mapping port 가 blocklist
- 6 14 phase phased rollout

— **14 phase phased rollout**. phase → → phase cutover. 4가: ① **port→IP 3** (In-memory Map + HASH + DB + manifest) — cold start ② **blocklist** — port_set(port port) + ip_set(IP), allocator 가 port lookup IP → ip_set hit port port_set 가 → **port rotation hazard** ③ **Adaptive Cooldown** — 5 outcome(success / block / timeout / networkError / authError / siteChange / unknownError) × consecutiveFailures × latency 가 health → ranked, Shadow mode ④ **Pool exhausted → legacy fallback** — exhausted legacy path, legacy blocklist IP skip(10). AGENTS.md — env value / / deprecated alias default. fallback AI 3 **follow-up** PR — AI

□□

- □ □□ **800□ → 90□ □ (88.75% □□)** — □□ □□□ □□.
- □□ □□□ **70% → 98%** — □□□ □□ □ □□ □□□□.
- □□ □□ SPOF □□(HA □□ □□ □□), □□□ □□□□ □□.
- □□ IP 가 legacy □□□ □□□□□ □□ **0□**.
- CROSSSLOT □□ **0□** — hash tag {pool} □□ □.
- ASN □□ □□ + KR 가 + □□ □□ □□(failed outcome □□) □□.

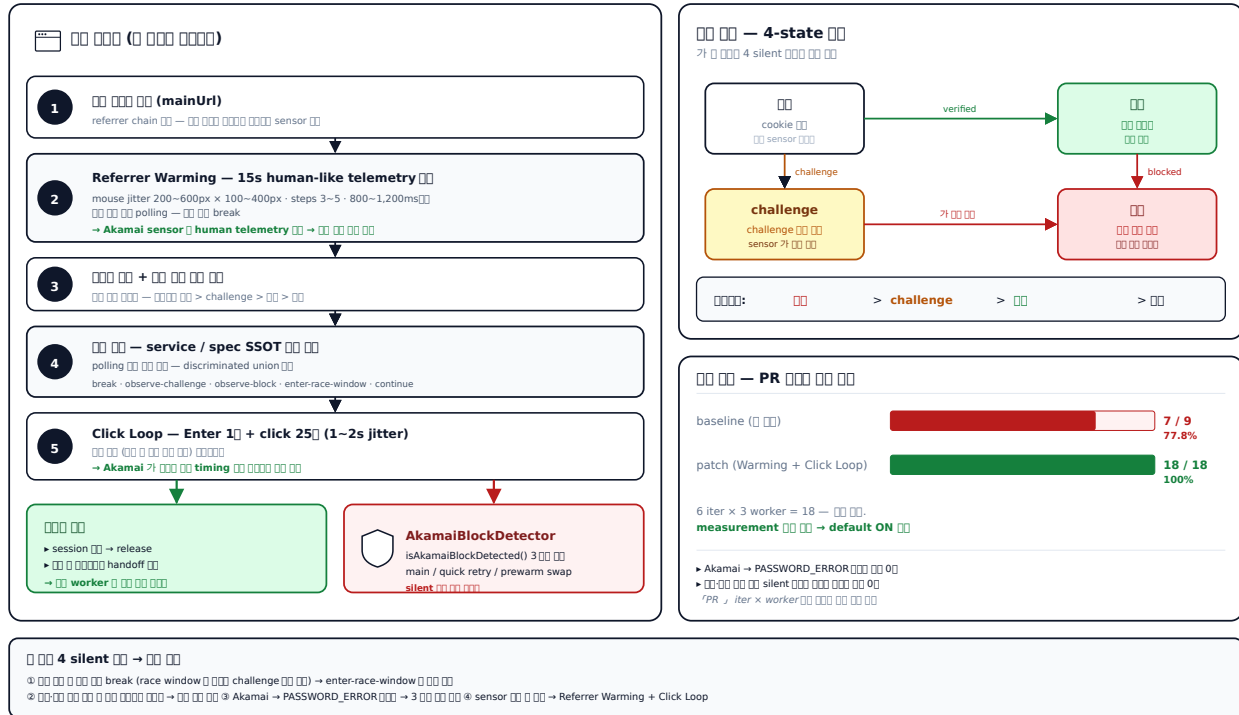
JVM/Spring □□□ ([commerce-external-gateway-kt](https://github.com/PreAgile/commerce-external-gateway-kt))

- [ADR-SCR-010 Bulkhead □
□](https://github.com/PreAgile/commerce-external-gateway-kt/blob/main/docs/adr/ADR-SCR-010-bulkhead.md) — Semaphore vs FixedThreadPool, □□ □□□□ □ □□
- [ADR-SCR-011 □□□□□ □□ □
□](https://github.com/PreAgile/commerce-external-gateway-kt/blob/main/docs/adr/ADR-SCR-011-decorator-order.md) — Bulkhead → CB → Retry → TimeLimiter → RateLimiter (Retry □□ CB □□ CB □□ N□ □□□□□ □□)
- WebClient ExchangeFilterFunction □□ proxy resolver □□□ + @ConfigurationProperties + @Validated □ env □□

Akamai Bot Manager — 4-state · Referrer Warming · Click Loop (77.8% → 100%)

C3. Akamai Bot Manager — 4-state · Referrer Warming · Click Loop

4-state 4 silent → 100% (18/18 iter)



C3. Akamai + 4-state

Akamai Bot Manager 가 99% stealth fork fingerprint sensor 가 pending sensor → 403.

4 가 silent:

- break → race window challenge
- sensor + 15s human-like telemetry
- Akamai application-level 401/403 password retry (Full Retry)
- sensor break → sensor 가 (1~2s) jitter

4-state 4 가 (a) 4-state (b) service spec SSOT (c) Referrer Warming — URL referrer chain, 15 human-like mouse jitter(200~600px × 100~400px × steps 3~5 × 800~1,200ms) → sensor 가 human telemetry → sensor (d) Click Loop — submit Enter 1 → page click 25 (1~2s jitter) (e) 3 — Akamai helper main loginResult + Quick Retry + Prewarm Swap → silent.

□□ — □ □□□(platformId)□ □□ □□ □□ N□□ □□□ □ □□□ □□□□ □□□□ □□ □□□□ □□□ (single-flight · coalescing).

- □ □□□ □□ — □-□□□ Map 80+ □□ □ □□□□ □□□. □□ □□□ □□□□ □□ □□ □□ / □□ □□ □□□ □□ □ □□ □□□ □
- □□□□□ □□ **invariant** □ — □□ □□□ (□□ cause □□ · □□ □□□□ □□) / 가 □(forceRelease □ □ record finally cleanup □ □ record □□ □□) / □□ throw □□□ (sync throw □ Map □□ □ throw)
- □□ □□□ spec □ □□□□ □□ □□□ □□ □ □□

□□ — **Port-Adapter(Hexagonal Architecture, Alistair Cockburn) + Decorator + Strategy** □

□ □□ OCP □□ + □□ □□□□□ **spec** □□ 7가 □□ □□□ spec □□□ □□□□□ describe □□□ 1:1 □□. Port □ execute / getInflightState / forceRelease □ □□, □□□□ in-process Map(forceRelease gate □□) + 4 □□□□□

(Deadline = Promise.race wall-clock timeout / Capacity = waiter cap load-shedding / Heartbeat = 15s scan long_running warn + gauge / Telemetry = owner_started/finished/failed/coalesced metrics) □□. 가 □ □□□.inflight.get(key) === record □□□ □□, sync throw □□□□

Promise.resolve().then(() => operation()) □ □□ □ hazard □□ □□. □□□ default □ deadline 180,000ms(2FA + waitForReplacement), maxWaiters 30(UI-spam load-shedding), long_running warn at 60s(stuck-owner early signal), forceRelease ops kill-switch.

□□ □□ 7가 — **spec** □□□ □□□ (□□□□ □□ □□ □□□ □□□□ □□□□ □□ □□□□ □□)

#	□□	□□
1	□□□□	□□ □ □□ □□□ □□□ 1□□ □□
2	□□ □□□	□□ cause □□□□ □□ (□□ □□□□ □□)
3	□□ □□	□□ settle □□□□ record □□ □□
4	kill switch	forceRelease □□ □□ □□□ □ owner, □□ □□
5	□□□ □□	□□ □□ □□ □□ □□
6	□□ □□	□□ throw □ promise rejection □□ □ □□
7	□□□	getInflightState □ entry □□ □□

□□

- □□□ □□ **80+** □ □□ — coordination + telemetry 가 thin call coordinator.execute(id, op, { telemetryTag }) □ □□.
- **22 new unit specs** — adapter/decorator □ □□ □□, □□ □□□ □□□ □□ □□.
- □□ □□ backend 가 □ □□□ □□ □ □□□□ DI factory □ swap.
- □□□ □□□□□ spec □□ □□□□ □□□□ □□ □□□ □□ □□□□□□ □□ □□□ □□□.

JVM/Spring □□□ ([commerce-external-gateway-kt](https://github.com/PreAgile/commerce-external-gateway-kt))

TS □□□□□ 7가 □□ □□□ Kotlin Coroutines □ □□□ □□.

- [ADR-SCR-008 Single-Flight Coordinator](https://github.com/PreAgile/commerce-external-gateway-kt/blob/main/docs/adr/ADR-SCR-008-single-flight.md) — Mutex + Deferred $\square$ in-process single-flight, 7-property $\square\square$ spec $\square\square$ $\square\square$
- Resilience4j Bulkhead.semaphore + TimeLimiter $\square$ deadline / capacity
- SF-2 $\square\square$ — thundering herd baseline vs Coroutines coalescing $\square\square$ **99.9% coalescing**
- MeterRegistry counter / gauge $\square$ singleflight.* telemetry (owner_started / finished / coalesced)

D. AI 環境 . 環境 環境

AI 기반의 다양한 서비스들이 등장하고 있습니다. 가령, TOCTOU 문제, 즉 cron 기반의 서비스 — AI 기반의 서비스도 등장하고 있습니다.

□□ □□ □□ - 6□□ □□□ □□ + 가 □ □□ □□·□□□ **OR** □□

00 — 00 0000 0000 00 0 0000 0:

- 5000 개 댓글 (sentiment 분석 — 약 30%가 5점 + 긍정 댓글)
- "ㅋㅋㅋㅋㅋㅋㅋㅋ" 100 개 댓글, 100 개 답글

가 00:

- AI 관련 정보 공유 및 ML 프로젝트 공유
- AI 관련 "가" 및 "100" 관련 정보 공유
- AI 관련 정보 공유 및 ML 프로젝트 공유

$\sigma = \sigma_0 + \sigma_1$ 가 if-else 6 가 σ_0 ML
shrink (Bell) **(Polarity Amplification)** + **tanh ±35 sigmoidal squashing**

□□□ □□ **6**□□

単語	単語
単語 単語 (top word share)	top_word_count / total_tokens
Bigram / Trigram 単語	most_common_ngram / total_ngrams
単語 単語	$-\sum p \log_2 p$ (単語 単語)
単語 単語	unique_chars / length (40 単語)
zlib 単語	len(compressed) / len(text)
単語 単語 / 単語 4+ 単語	単語 単語 単語

가까이 0.0 noise. 가 0 Bell shrink + weight tanh ±35 box squash. rating ≤ critical_rating OR 5 + score in (min~max) + len ≥ 10 scoring-based. BIZ(send_negative_alarms` + `critical_review_rating` ) User(NegativeReviewAlarm.is\_active`) , SCORE_VERSION PK review_id → v2 v1

□□

- AI □□ □□ — existing_scores_map pre-fetch □ □□ □□ □□ □□ + □□□ □□ □□□ □□ ML □□ skip.
- 5□ □□ □□ □□□ □□ — □□ + □□□□ OR □□□□ □ 30% 가 □□ ≠ □□□ □□.
- □ □□ □□□ □□ BIZ 3□ + User 3□ = 6□ □□□ □□ □□.
- □□□ □□□ □□ □□ — □□ □□□(□□□·□□) + □□□□ □□ □□ Enum □□ □□.

JVM/Spring □□□ ([commerce-batch-orchestrator](https://github.com/PreAgile/commerce-batch-orchestrator))

- Spring Batch ItemProcessor □□ — 6□□ □□□ □□ + 가 □ □□ AI Score □□
- WebClient (□□□) □ □□ MLOps Score API □□, Bean Validation □□ □□ □□
- Kafka Consumer Group + DLQ + retry topic □□ □□ □□ □□ □□□□ □□□□□
([ADR-MQ-008](https://github.com/PreAgile/commerce-batch-orchestrator/blob/main/docs/adr/ADR-MQ-008-consumer-idempotent.md))
- ApplicationEventPublisher + @TransactionalEventListener(phase=AFTER_COMMIT) □□ □□ □□
□ □□□□ □ □□

SafeCron — □□ □□□□ cron □□ □□ □□ □□□□□ (14□ □□)

□□ — □□ □□□□ □□□□ □□ @Cron □ □□ □□□□□□ □□ □□.

- □□ □□ □□□ □□□ N□ □□□□□□ N□ → □□ PG API rate-limit □□
- □□ □□□ cron N□ → □□□□ □□ □□□ N□
- □□□ □□□ □□ N□ → □□ SaaS rate-limit + □□

□□ □ □□ □:

- □□ cron □□ □ □□ □□ = □□□□□□□ □□
- □ □□ □□ □ □□□□ □□□ □ □□ □□ → cron □□ □ □

□□ — □□□□□ + **DI** □□ □□ □□ □ □ □□□□□□ □□ □ + Slack □□□ + □□□□ □□□ □□□. □□□□□□ □□□ □□ □□□ □□□ □□
가 □□□□, □□ □□□ ModuleRef □□ □□□ □□□ □ □□□□□ □□□ □□ □□ □□□□ lazy □□□□□. □ TTL □□ □□□ OOM /
SIGKILL □□□ □□ cron □□ □□. □□□□□ Slack □□□ □□ — start / complete (\${duration}ms) / error - \$
{message} + □□ trace block — 가 □ □□□□□. slackAlert: false □ □□□□□ cron □□ □□, slackWebhookUrl
□ cron □ □□ □□ □□□. CRON_ENABLED □□□□□ □□□ □□□ □□□□□ □□ □□□□ □□ □□ → □□□□ □ □ → QA □□.

□□

- □□□ **14□** (subscriptions, □□□, □□□ SaaS □□□, magic-link, mixpanel, log-cleanup □).
- □□ 5~10□□ □□□□□□□ □□ → □ **100□□** □□ + cron 가 □□□ □□ □ □□□□□.
- □□□□ □ □ □□□ □□ cron □□□□ — QA □□ □□ □□□□ □□.
- DI □□ + Lock acquire/release/TTL + Slack □□□ + □□□□ □□□ □ □□□□□□ □□ — □□ □ □□□□□□□ 5~10□
□ □□.

JVM/Spring □□□ ([commerce-batch-orchestrator](https://github.com/PreAgile/commerce-batch-orchestrator))

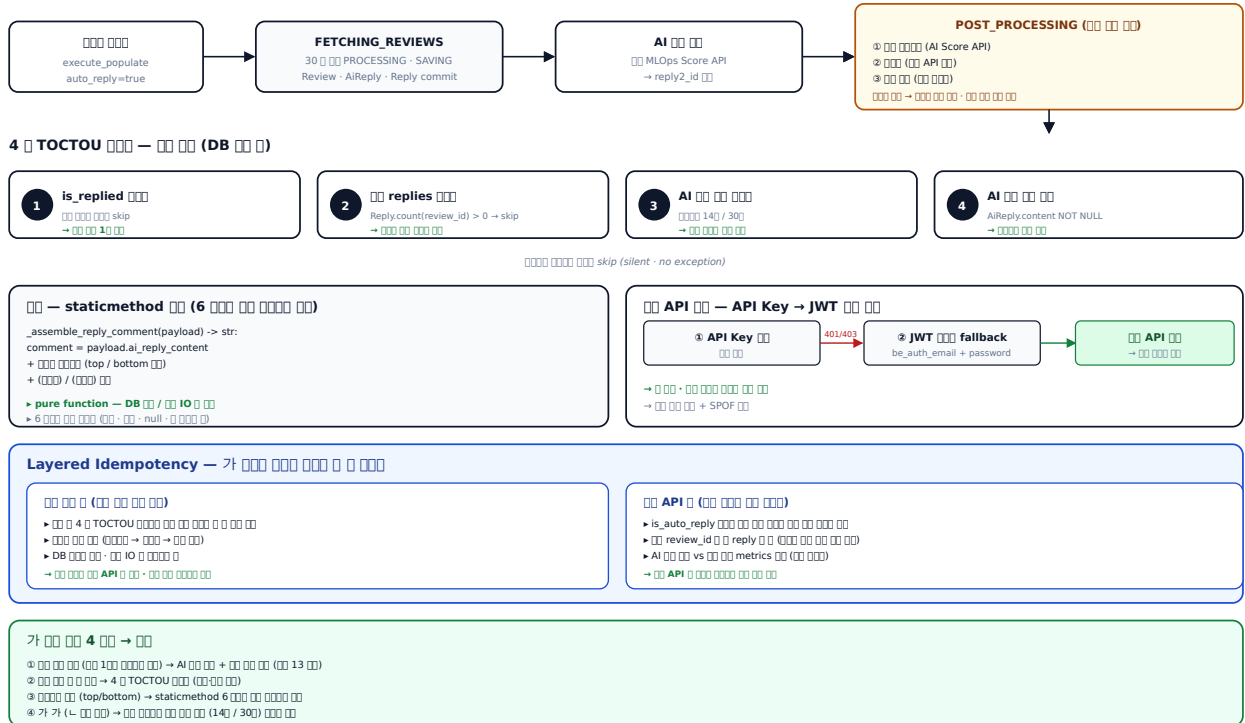
- Spring @Scheduled + **ShedLock** (Redis / JDBC backend) — □□ □□□□□ cron □□ □ □□ □□ +
ADR-005 Outbox polling-to-CDC □□ □□

- @ConditionalOnProperty 이 toggle (QA 이)
- Spring AOP @Around 이 Slack 이 / 이 / 이 이 이

이 이 이 이 — 이 → AI → 4 TOCTOU 이 → API Key/JWT 이 이 이

D1. 이 이 이 이 — 이 → AI → 4 TOCTOU 이 → 이 이 이

이 이 이 이 + API Key → JWT fallback + Layered Idempotency 이 이 이



D1. 이 이 이 이 — TOCTOU 4 이 + 이 이

이 — 이 이 이 이 AI 가 이 이 이 4 가:

- 이 이 이 — 이 1이 "이 이 이 이" 이 = 이
- 이 이 — 이 이 이 이 이
- 이 이 이 이 — 이 이 이 이 AI 이 이 / 이 이
- 이 이 이 이 — 이 이 이 5이 / 4이 이 이 이

가 DB 이 이 60가 이 이 이 API 이 이 이 이 이 이 이 이.

이 — 이 이 이 이 + **API Key → JWT 이 이 fallback + TOCTOU 4** 이 이 이 3이 이. payload 이 이 DB 이, 이 staticmethod, 이 async 이 3가 이 이 이 DB 이 이 이 (Spring @Transactional 이 이 이 이). 4 TOCTOU 이 ① is_replied 이 ② 이 replies 이 ③ AI 이 이 ④ AI 이 이 이 — 이 이 이 이 skip. 이 staticmethod 이 이 6가 이 이 이 이. 이 API 이 API Key 이 → 401/403 이 이 JWT 이 이 이, 이 이 · 이 이 이 이 이 이. 이 이 이 이 — 이 이 이 → 이 이 → 이 이 (가 이 이, 이 이 이 이 이 이 이 이 이 이 이 이 이 이).

Layered Idempotency 이 이

- 이 이 이: 이 이 4 이 이 이 이 이 이 이 이 (이 이 이 이 이)

